Pigment, wet media, and canvas feedback: research routes

This is exploratory advice for a future all-brush graph, **not** a Painter design decision or an implementation claim. Painter currently saves ordered immutable document-space sampled strokes and deterministically derives pencil splats; its 512-pixel multi-resolution raster tiles cache the result, including incremental additive pencil updates. No pigment transport, wetness field, smudge sampler, paint-height field, or brush graph exists today. The proposed operations below would change both the evaluation and persistence contracts. The project brief's aspiration that strokes remain resolution-independent mathematical objects does not, by itself, make a grid simulation resolution-independent.

Separate color optics from material motion

Documented optics. Kubelka and Munk derive reflection of diffusely illuminated matte coatings from absorption and scattering; thickness and substrate reflectance matter for a finite coating, while their original treatment explicitly does not cover glossy coatings [1]. Per-wavelength absorption $K(\lambda)$ and scattering $S(\lambda)$ of measured pigments can be combined by concentration and used to predict spectral reflectance before illuminant/observer conversion to display RGB [2, 3]. This is a route to **pigment color mixing**, not a model of paint flowing, paper absorbing water, brush pickup, or oil gloss. The measured palette, binder, substrate, illuminant, scattering assumptions, layer thickness, and display gamut determine how faithful the result can be; a fixed display RGB is not a unique spectrum [2, 3].

Two distinct color routes to investigate. (A) Track pigment quantities or spectral coefficients and paint thickness per simulation cell; render a mixture and, where needed, finite-thickness layers against the underlying substrate. This affords material-specific calibration and optical glazing but multiplies storage and work by pigments or wavelength samples; in the IMPaSTo demonstration pigment layers and eight selected wavelengths support interactive rendering rather than an unrestricted full-spectrum solver [3]. (B) Mixbox encodes arbitrary RGB colors as a four-pigment latent plus an additive RGB residual, averages latents, and decodes RGB with lookup tables; the paper reports two 48 MB runtime lookup tables, and its limitations explicitly include four primaries, homogeneous opaque mixing, and no faithful translucent-layer compositing [2]. Its pigment-informed RGB-in/RGB-out appearance is a useful **visual approximation**, not conserved physical pigment masses or proof that any input RGB corresponds to a real pigment. Never equate alpha compositing, RGB interpolation, Mixbox-style mixing, and finite-thickness Kubelka–Munk layering [1, 2].

Wet behavior is a separate route. Curtis and colleagues model surface watercolor flow, pigment movement and paper absorption/backruns with optical paint layering [4]; Chu and Tai's MoXi instead emphasizes percolation through absorbent paper, variable permeability and boundary pinning, plus pigment supply/advection/fixation [5]. MoXi explicitly warns that Fickian diffusion alone merely smooths concentration and misses the branching and flow patterns it targets [5]. Diffusion can cheaply soften a mark; advecting pigment with water motion, absorption into paper, boundary pinning and drying can yield qualitatively different effects. Do not label Gaussian blur or noisy edge darkening a physically simulated watercolor. Oil-like impasto is a different problem: IMPaSTo transports paint volume and pigments conservatively in a 2.5D height-field, uses brush motion/penetration heuristics and bidirectional brush–canvas transfer, and keeps one active wet layer with accumulated dry layers [3]. Its own authors distinguish those approximations from full non-Newtonian free-surface physics [3].

Feedback and typed candidate operations

Proposed vocabulary, not existing nodes. Distinguish `SampleStroke` (document-space points, pressure, time), `StampMask` (coverage/shape), `ReadCanvas` (a snapshot of a specified material state and region), `ReadBrushReservoir` (brush-loaded color/water/pigment), `MixColor` (explicit RGB, latent-pigment, or spectral rule), `TransferPigment` (brush ↔ canvas quantities), `ApplyFlux` (neighbor-to-neighbor material transport), `Deposit` (write material), `Dry` (water loss and/or pigment fixation), and `Shade` (material state → display pixels). Outputs should be typed by role and units—for example `ColorRGB` versus `PigmentMass[]`, `WaterAmount`, `PaintVolume`, `Velocity`, `Coverage`, `DocumentRect`, `Duration`—so a color interpolation cannot masquerade as mass transport. Names and types here are research probes, not an API commitment [2, 3, 5].

Graph boundaries worth comparing.

- Hard digital: `SampleStroke → StampMask → Deposit(ColorRGB, Coverage) → Shade`. No canvas read or evolving medium; preserve a cheap deterministic path for the whole all-brush ambition.
- Plausible smudge: `SampleStroke → ReadCanvas(RGB snapshot at preceding dab) → MixColor(sample, brush color, strength) → Deposit(RGB)`. A dab-to-dab feedback edge belongs to a **time-step or ordered-dab boundary**, not an unrestricted combinational cycle; sampling the already-written current dab is ambiguous. RGB is moved/copied visually here, not a conserved pigment quantity [6].
- Material pickup/deposit: `ReadCanvas(pigment masses, volume) + ReadBrushReservoir → TransferPigment → (new canvas, new reservoir) → Shade`. A transfer debit and credit should match before fixation or deliberate loss; merely reading a composited image cannot recover its hidden constituent pigments [3].
- Wet wash: `StampMask → Deposit(water, pigment) → [absorb → transport → pin/fixture → evaporate] per step → Shade`. Read neighboring cells from one frozen step, write the next, and repeat until dry or explicitly paused; brush dabs need not be the only source of state change [4, 5].
- Oil-like: `StampMask + ReadCanvas(height, pigment) + ReadBrushReservoir → push/transfer → (wet height, pigment, reservoir) → Shade(height, reflectance)`. Geometry, optical mixture, and post-contact drying are separate concerns [3].

The second through fifth examples require feedback owned by the canvas/simulation timeline, not a purportedly pure per-point stroke function. An acyclic authoring graph could still contain an explicit `PreviousStep`/`NextStep` state boundary; the important questions are which state is readable, who owns the clock, and how a stroke's effects are ordered. If brush graphs can read other canvas nodes or a flattened composition, snapshot selection and dependency invalidation must be explicit: a downstream edit can otherwise silently change the color picked up by an old stroke. Krita's manual warns of exactly this user-facing issue when its smudge brush samples an all-layers overlay [6].

Simulation accounting versus a convincing picture

Proposed Painter conservation rules: source accounting should distinguish material injected from the brush reservoir from material picked up and moved within a wet layer; local pigment should not spontaneously increase under diffusion or advection; drying should change solvent or binder state, not create pigment; and a clean pickup-only stroke should be able to remove material from one area while depositing it elsewhere. This proposed invariant set draws its paint-volume and pigment-mass transport inspiration from IMPaSTo's conservative advection [3], while MoXi supplies a precedent for separate water, mobile pigment and fixed pigment state and their exchange [5]. Neither paper enforces this exact Painter model.

For **appearance-only** media, a local color pickup reservoir, controlled bleed, texture, edge darkening and delayed settling may be more valuable than a fluid solver if artist control and latency

dominate. Call these perceptual effects, not water/pigment simulation. Stam's semi-Lagrangian method permits larger stable fluid steps but has numerical dissipation [7]; conversely, IMPaSTo's flux-based transport preserves moved paint and constrains displacement per cell/step [3]. Stability, conservation, sharp fronts, and speed are distinct properties. This is a testable comparison, not a decree to use either scheme.

The tiled, replayable document is the hard interface

The present cache can incrementally paint a new pencil stroke onto its old image because pencil deposits only add alpha. A smudge samples earlier rendered color, a wet wash evolves after input stops, and a pushed oil stroke changes nearby old marks; none is generally reconstructible from `appliedStrokes` plus only the stroke's local splats. A possible research contract is to preserve immutable authored **commands** (stroke samples, brush graph/version, deterministic seed, material/time commands) while storing derived simulation snapshots or tile checkpoints as disposable/rebuildable data. But replay from the beginning, checkpoints plus ordered replay, and persistently storing material fields offer different tradeoffs in startup, memory, editability and exact undo. Do not promise unlimited-resolution replay from a single fixed grid.

- **Sampled reads and synchronization:** define whether pickup reads the active paint node's wet fields, its dried render, or the composite of other nodes; use a stable pre-dab or pre-step snapshot and serialize dependent writes. A visible low-res tile or stale fallback is suitable for drawing a preview, not necessarily for authoritative sampling. Brush reservoir state must also be replayed across dabs and strokes, with explicit refill/clean/reset rules.
- **Geometry and seams:** a transport cell near a tile boundary reads neighbors and writes fluxes across that boundary. Own each shared edge once, exchange halos/ghost cells from the same step, and include support radius plus any flow/diffusion reach in invalidation; a raster-only bleed margin cannot guarantee mass balance. A wet area may expand outside the originally touched tiles, including while idle; dirty-region bounds and active-step scheduling must follow the evolving state. Flux and halo behavior at canvas edges need a defined boundary condition.
- **Resolution:** record units in document space, solver cell size, step duration, paper texture scale, random seed and upsampling method. A coarser preview and a fine export are different discretizations; re-solving may change edge pinning, pigment quantities per sampled cell and stroke pickup. MoXi reports simulation at 512^2 with a 3× image-space detail pass, explicitly as quality enhancement rather than fine-scale simulation [5]. Decide whether exports reproduce the committed simulation exactly, or deliberately recompute a different-resolution interpretation; disclose the difference to artists.
- **Undo/replay:** removing a historical stroke requires re-evaluating every later command that read or transported its state, potentially beyond its original bounds. Checkpoints may limit the recompute interval but cannot make later dependencies vanish. Undo a time-advanced drying event or a brush-reservoir transfer as an ordered command or restore a compatible checkpoint; include graph version, color model, solver settings, source images and a deterministic tick sequence if cross-session replay fidelity is promised. IMPaSTo used 64×64 dirty/undo tiles and a GPU-local undo texture for its raster model; that is a useful alternative to evaluate, not evidence that Painter's current pencil cache supports simulation undo [3].

Two useful finds outside the default watercolor/oil citations

- **Krita's Color Smudge Brush Engine** documents two distinct modes: Smearing copies from the preceding brush position, whereas Dulling samples beneath the dab and fills its shape before color/opacity are applied. Dab spacing changes their effects differently; its Overlay option samples all layers and can make later edits to lower layers surprising. Relevance: this supplies concrete

artist-facing controls and tests for read scope, feedback timing, and whether a smudge is copy, average, or actual material transfer; no assertion that Krita solves pigment physics follows [6].
- **MoXi's real-time ink system** couples porous-flow simulation to deliberately nonphysical artistic controls (rewet, moisture redoubling, ink push/suck), and derives high-resolution boundaries/textures from a lower-resolution flow field [5]. Relevance: a graph can expose simulation controls without pretending all user operations are literal physics; it also provides a specific coarse-simulation-versus-detail-rendering benchmark and the danger of conflating the two.
- **Mixbox's Rebelle integration** demonstrates a pigment-informed RGB-in/RGB-out mixing layer used alongside other media behavior, rather than requiring the flow solver and optics to be one subsystem. Relevance: a focused optical-mixing experiment could be evaluated independently before committing to per-pixel physical pigments; its four-pigment and opaque-mixture limits must remain visible [2].

Open questions and staged experiments

- What artist-visible outcome is primary: intuitive blue–yellow mixes, controllable smudge, watercolor backruns, ink branching, thick relief, or calibrated spectral matches? Which actions must work on a composited canvas rather than one paint node? What is an acceptable difference between preview and export?
- Are brush colors arbitrary RGB, a measured pigment palette, or both? Are washes translucent layers requiring substrate-aware finite-thickness optics? What fields and precision are affordable per active document-space cell and per brush reservoir? What exactly persists in a saved document?
- Should wetness advance in real time while idle, only on explicit ticks, or through a replayable time command? What if the app is closed while a wash is wet? What does erasing or reordering a stroke mean after downstream pickup?

Suggested evidence ladder, not a rollout commitment. First compare ordinary RGB blending, Mixbox-style latent RGB mixing, and a few measured spectral mixtures on the same swatches with explicit illuminant, substrate and opacity assumptions [1, 2, 3]. Next prototype a snapshot-based RGB smudge with separate pickup/color/deposit rates and a dab-order/undo fixture; compare Smear versus Dulling interactions without claiming physical pigment transport [6]. Then use a small fixed-resolution wet-paper sandbox with mobile/fixed pigment and water, no cross-tile shortcuts: compare diffusion-only, flux advection and permeability/pinning on identical inputs, recording mass/water budgets, visible fronts, step cost, and dry/rewet results [3, 5, 7]. Only if those trials justify it, investigate cross-tile halos/checkpoints, multiple resolution replay, and oil height/brush-reservoir coupling in the actual document/cache workflow. Record the perceptual benefit **and** whether undo, seams, idle evolution and export remain acceptable before promoting any route into a graph contract.

Primary sources

[1] Paul Kubelka and Franz Munk, "An Article on Optics of Paint Layers" (1931; English translation), Cornell-hosted paper.

[2] Šárka Sochorová and Ondřej Jamriška, "Practical Pigment Mixing for Digital Painting," ACM Transactions on Graphics 40(6), 2021, authors' paper publisher DOI.

[3] William Baxter, Jeremy Wendt, and Ming C. Lin, "IMPaSTo: A Realistic, Interactive Model for Paint," NPAR 2004, authors' paper.

[4] Cassidy J. Curtis, Sean E. Anderson, Joshua E. Seims, Kurt W. Fleischer, and David H. Salesin, "Computer-Generated Watercolor," SIGGRAPH 1997, authors' paper publisher DOI.

[5] Nelson S.-H. Chu and Chiew-Lan Tai, “MoXi: Real-Time Ink Dispersion in Absorbent Paper,” ACM Transactions on Graphics 24(3), 2005, authors’ paper publisher DOI.

[6] Krita project, Color Smudge Brush Engine manual, sections “Smudge Length,” “Overlay,” and “Paint Thickness.”

[7] Jos Stam, “Stable Fluids,” SIGGRAPH 1999, paper publisher DOI.